

Recitation 5

Timers in Embedded Systems

Table of Contents

1. [Introduction — Why Timers?](#)
 2. [Types of Timers](#)
 3. [Timers on the STM32L475](#)
 4. [Mbed OS Timer API Overview](#)
 5. [Timer — Elapsed Time Measurement](#)
 6. [Interrupts and Callbacks — Just Enough](#)
 7. [Timeout — One-Shot Timer](#)
 8. [Ticker — Periodic Timer](#)
 9. [LowPowerTicker — Periodic Timer Across Sleep](#)
 10. [References](#)
-

1. Introduction — Why Timers?

Timing is fundamental to embedded systems. Almost every real application needs to measure elapsed time, trigger an action after a delay, run a task at a fixed rate, or drive a signal at a precise frequency. The question is how you implement that in software.

The Busy-Wait Problem

```
// DO NOT use this in real code
void delay_ms(int ms) {
    volatile int count = ms * 8000; // rough cycle count, not reliable
    while (count > 0) count--;
}
```

This works in the simplest of cases, but it has serious problems:

- CPU is stuck in the loop doing nothing else
- You cannot read sensors, handle communication, or do any other work during the delay
- The count depends on clock speed and compiler settings — not accurate

Hardware Timers as Independent Peripherals

A hardware timer is a peripheral — a separate circuit on the chip that counts independently of the CPU. You configure it, start it, and it runs on its own. The CPU is free to do other work. When the timer reaches its target count, it can:

- Set a flag that the CPU checks when convenient (polling)

- Interrupt the CPU immediately to run a handler function

This is the foundation of everything in this recitation. Every Mbed timer object described below is a software abstraction sitting on top of one or more hardware timer peripherals.

2. Types of Timers

Timers can be categorized in two ways: by the clock source that drives them, and by their functional behavior.

By Clock Source

High-Speed External Crystal (HSE) An external crystal (typically 8 MHz) connected to the chip. Very accurate. The STM32L475 uses this as the source for its PLL (Phase-Locked Loop) to reach 80 MHz system clock.

High-Speed Internal RC (HSI) Built-in RC oscillator at 16 MHz. No external component needed but drifts with temperature. Good enough for most tasks, not ideal for precision timing.

Low-Speed External Crystal (LSE) A 32.768 kHz crystal. This specific frequency (2^{15}) divides cleanly to 1 Hz. Designed to keep running during deep sleep at under 1 μ A. Feeds the RTC and LPTIM on the STM32L475.

Low-Speed Internal RC (LSI) Built-in ~32 kHz oscillator. Serves the same purpose as LSE but less accurate (± 10 – 20%). Available even if no LSE crystal is on the board.

Independent Watchdog (IWDG) Runs off LSI. If your firmware does not periodically reset ("kick") this timer, it resets the microcontroller. Used to recover from software hangs.

By Functional Behavior

One-Shot Counts once, fires, stops. Example: turn off an LED 2 seconds after turning it on.

Periodic Automatically reloads and repeats. Example: toggle an LED every 500 ms.

Input Capture Records a timestamp when an external signal changes. Used to measure pulse width or frequency of an incoming signal.

PWM Output (Output Compare) Drives a GPIO high and low at a set frequency and duty cycle. Used for motors, servos, LED dimming.

Watchdog System safety timer — resets the MCU if firmware stops responding. Not part of the Mbed Timer API in this recitation.

3. Timers on the STM32L475

Important distinction first

The timer peripherals (TIM1–TIM8, LPTIM1/2, SysTick, RTC) are all inside the STM32L475VG chip. The development board itself does not add any timer hardware.

What the board adds are the **external clock components** that feed those internal timers. There are three clock positions on the B-L475E-IOT01A (user manual, Section 7.6):

X2 — 32.768 kHz LSE Crystal (fitted on the board) Part number NX3215SA-32.768K, connected to PC14/PC15. This is the physical crystal you can see on the board next to the STM32 chip. It runs continuously even during deep sleep and feeds LPTIM1/2 and the RTC. Without this crystal, **LowPowerTicker** falls back to the less accurate internal LSI oscillator.

X3 — 8 MHz from ST-LINK MCU (fitted) Instead of a dedicated 8 MHz crystal, the board routes the clock output from the ST-LINK chip (STM32F103CBT6) to PH0/PH1 of the main MCU. This is the HSE source. The STM32L475 multiplies it to 80 MHz through its PLL to run the system clock and all general-purpose timers.

X1 — 8 MHz HSE crystal footprint (not fitted) The PCB has a footprint for a dedicated 8 MHz crystal at X1 (also capacitor pads C2 and C4), but they are not populated. On a custom design you would use X1 instead of relying on the ST-LINK clock. For this board, X3 covers it.

Note: If you unplug the ST-LINK in a battery-powered deployment, the STM32L475 loses its HSE source and falls back to HSI at 16 MHz. The LSE crystal X2 keeps running regardless, so **LowPowerTicker** still works.

Timer Peripherals Inside the STM32L475VG

STM32L4+ Reference Manual (RM0432) — General Purpose Timers: Page 1298/2300

General Purpose Timers — TIM2, TIM3, TIM4, TIM5 STM32L4+ Reference Manual Page no:1298/2300 in PDF

- TIM2 and TIM5 are 32-bit; TIM3 and TIM4 are 16-bit
- Each has 4 capture/compare channels that can connect to GPIO pins
- Support PWM output, input capture, one-pulse mode, encoder interface
- Clocked from APB1 at up to 80 MHz

Mbed uses these internally for **Ticker** and **Timeout**. Their channels are also exposed on the Arduino connector pins — TIM3_CH1 on D5, TIM3_CH3 on D3, TIM3_CH4 on D6, TIM2_CH1 on D9, TIM2_CH3 on D10.

Advanced Control Timers — TIM1, TIM8

Everything the GP timers have, plus complementary outputs with dead-time insertion and a break input for emergency shutdown. Designed for motor control. Mbed does not expose these features directly — use the STM32 HAL if you need them.

Basic Timers — TIM6, TIM7

Counter only, no GPIO channels. Used to trigger the DAC or as a general time base. Mbed uses TIM6 as the **us_ticker** — the 1 μ s tick source that all **Timer**, **Timeout**, and **Ticker** objects read from.

Low Power Timer — LPTIM1, LPTIM2

Designed to keep counting during Stop and Standby sleep modes. Clocked from the LSE crystal X2 at 32.768 kHz (~30.5 μ s per tick). Draw only a few μ A during sleep. Can wake the CPU when they expire. **LowPowerTicker** and **LowPowerTimeout** in Mbed sit on top of LPTIM.

SysTick

Part of the ARM Cortex-M4 core, not an STM32 peripheral. Generates a 1 ms interrupt that drives the Mbed OS RTOS scheduler. Do not reconfigure it — Mbed owns it.

RTC — Real-Time Clock

Internal peripheral clocked from LSE crystal X2. Keeps date and time through deep sleep and power cycles (if VBAT is connected). `LowPowerTimer` in Mbed uses the RTC to measure elapsed time that survives sleep.

Board vs Chip — Summary

Component	Location	What it provides
X2 (32.768 kHz crystal)	On the PCB	LSE → RTC, LPTIM1/2
X3 (8 MHz from ST-LINK MCU)	On the PCB	HSE → PLL → 80 MHz system clock
X1 (8 MHz crystal footprint)	On the PCB, not populated	Would replace X3
HSI (16 MHz RC)	Inside STM32L475	Fallback if HSE not available
LSI (~32 kHz RC)	Inside STM32L475	Fallback if no LSE crystal
TIM1–TIM8	Inside STM32L475	GP, advanced, and basic timers
LPTIM1, LPTIM2	Inside STM32L475	Low-power timers
SysTick	Inside ARM Cortex-M4 core	RTOS tick
RTC	Inside STM32L475	Calendar, elapsed time across sleep

Hardware Timer to Mbed Object Mapping

Hardware	Clock Source	Board Component	Mbed Object
TIM6 (us_ticker)	APB1 / 80 MHz	X3 → PLL	<code>Timer</code> , <code>Timeout</code> , <code>Ticker</code>
LPTIM1 / LPTIM2	LSE 32.768 kHz	X2 crystal	<code>LowPowerTicker</code> , <code>LowPowerTimeout</code>
RTC	LSE 32.768 kHz	X2 crystal	<code>LowPowerTimer</code>
SysTick	CPU core clock	X3 → PLL	RTOS kernel (do not touch)
TIM1–TIM5, TIM8	APB1/APB2 / 80 MHz	X3 → PLL	<code>PwmOut</code> , HAL only

4. Mbed OS Timer API Overview

Mbed provides five timer objects. Here is a quick summary before we go into each one.

Object	What it does	Interrupt-driven	Survives sleep
Timer	Measures elapsed time	No (you check it)	No
Timeout	Calls a function once after a delay	Yes	No
Ticker	Calls a function repeatedly at a fixed rate	Yes	No
LowPowerTimeout	One-shot, works during sleep	Yes	Yes
LowPowerTicker	Repeating, works during sleep	Yes	Yes

The main split: **Timer** is polling-based — you read it yourself. Everything else calls a function automatically when the time comes, which requires understanding interrupts first. That is Section 6.

5. Timer — Elapsed Time Measurement

What is it?

Timer is a stopwatch. Start it, do things, read back how much time has passed. It does not call any function on its own — you check it manually when you need it.

Under the hood it reads from TIM6 (the `us_ticker`) at 1 μ s resolution.

When to use it

Use Case	Timer?
Measure how long a function takes	Yes
Run a task periodically by checking elapsed time	Yes
Debounce a button	Yes
Check if a state has lasted longer than N seconds	Yes
Do something after a delay without blocking main	Use Timeout instead
Blink an LED at a precise rate	Use Ticker instead

API

```
Timer t;

t.start();           // start counting
t.stop();            // pause
t.reset();           // reset to zero
t.elapsed_time();    // returns chrono::microseconds (Mbed OS 6 style)
t.read_ms();         // returns int milliseconds (older style, still works)
```

Polling Pattern

You can build a periodic task using `Timer` without any interrupts:

```
#include "mbed.h"

Timer t;
DigitalOut led(LED1);

int main() {
    t.start();
    while (true) {
        if (t.elapsed_time() >= 500ms) {
            led = !led;
            t.reset();
        }
        // other work here
    }
}
```

This works fine as long as the "other work" is fast. If it takes 600 ms, the LED toggles late. That is exactly why `Ticker` exists — covered in Section 8.

Exercise 1 — Measure Elapsed Time and Print to Serial

```
#include "mbed.h"

int main() {
    Timer t;
    int iteration = 0;

    printf("Timer demo starting\n");
    t.start();

    while (true) {
        iteration++;
        ThisThread::sleep_for(200ms); // simulate work

        // read elapsed time using chrono
        auto elapsed = t.elapsed_time();
        int elapsed_ms = chrono::duration_cast<chrono::milliseconds>
(elapsed).count();

        printf("Iteration %d | Elapsed: %d ms\n", iteration, elapsed_ms);

        if (iteration % 5 == 0) {
            printf("--- Resetting timer ---\n");
            t.reset();
        }
    }
}
```

Expected output:

```
Timer demo starting
Iteration 1 | Elapsed: 200 ms
Iteration 2 | Elapsed: 401 ms
Iteration 3 | Elapsed: 602 ms
Iteration 4 | Elapsed: 803 ms
Iteration 5 | Elapsed: 1004 ms
--- Resetting timer ---
Iteration 6 | Elapsed: 201 ms
```

6. Interrupts and Callbacks

`Timeout`, `Ticker`, and `LowPowerTicker` all call a function automatically when the timer fires. To use them correctly you need to understand how that works.

What is an Interrupt?

Normally the CPU runs through your code line by line. An interrupt is a hardware signal that forces the CPU to pause, run a special function (the ISR — Interrupt Service Routine), and then return exactly where it left off.

Normal execution:

```
main() —————>
      line1, line2, line3 ...
```

Timer fires:

```
main() ————| PAUSE |—————>
              |       |
              v       ^
            ISR runs ———|
            (must be short and fast)
```

The main program does not know the ISR ran — from its perspective, a few microseconds just disappeared.

What is a Callback?

A callback is a function you write and hand to Mbed to call later. With `Ticker`, you pass your function to `attach()`. Mbed stores it and calls it from the ISR every time the timer fires.

```
void my_callback() {
    led = !led; // this runs inside an ISR
}

ticker.attach(&my_callback, 500ms);
// you never call my_callback() directly – Mbed calls it for you
```

ISR Safety Rules

The ISR runs at any moment, interrupting your normal code. This creates restrictions.

NOT safe inside a Ticker or Timeout callback:

- `printf()` — uses a mutex internally, will deadlock or corrupt serial output
- `malloc()` / `new` — heap allocation is not interrupt-safe
- `ThisThread::sleep_for()` — you cannot sleep inside an ISR
- Any Mbed RTOS call that uses mutexes, semaphores, or queues
- Anything that takes more than a few microseconds

Safe inside an ISR:

- Toggling a `DigitalOut`
- Reading/writing `volatile` variables
- Setting a flag (`volatile bool flag = true`)

The rule: do the minimum in the ISR, handle the real work in main.

The Flag Pattern

Standard way to communicate from an ISR to main:

```
volatile bool flag = false;

void on_ticker() {
    flag = true; // safe — simple write
}

int main() {
    ticker.attach(&on_ticker, 500ms);
    while (true) {
        if (flag) {
            flag = false;
            printf("Ticker fired\n"); // safe — we are in main now
        }
    }
}
```

7. Timeout — One-Shot Timer

What is it?

`Timeout` calls a function once after a delay, then stops. Your main program keeps running during the delay — it does not block. The callback fires automatically when time is up.

When to use it

Use Case	Timeout?
Turn off an LED N seconds after turning it on	Yes
Trigger a retry if no response arrives in 3s	Yes
Detect if a state has been held too long	Yes
Repeat something at a fixed rate	Use Ticker instead
Measure duration	Use Timer instead

API

```
Timeout t;

t.attach(&my_callback, 2s);    // fire once after 2 seconds
t.attach(&my_callback, 500ms); // fire once after 500ms
t.detach();                   // cancel before it fires
```

Concept Snippet

```
#include "mbed.h"

DigitalOut led(LED1);
Timeout t;

void turn_off() {
    led = 0; // ISR-safe
}

int main() {
    led = 1;
    t.attach(&turn_off, 2s); // LED turns off automatically after 2s
    while (true) {
        // main keeps running – no blocking
    }
}
```

Exercise 2 — LED On, Auto-Off After 2 Seconds

```
#include "mbed.h"

DigitalOut led(LED1);
Timeout off_timer;
```

```
volatile bool led_off_event = false;

void on_timeout() {
    led = 0;           // turn off LED – ISR-safe
    led_off_event = true; // flag for main to print
}

int main() {
    printf("Timeout demo starting\n");

    led = 1;
    off_timer.attach(&on_timeout, 2s);
    printf("LED on. Will turn off in 2 seconds.\n");

    int loop_count = 0;

    while (true) {
        loop_count++;
        ThisThread::sleep_for(200ms);
        printf("Main loop iteration %d\n", loop_count);

        if (led_off_event) {
            led_off_event = false;
            printf("LED turned off by Timeout.\n");
        }
    }
}
```

Expected output:

```
Timeout demo starting
LED on. Will turn off in 2 seconds.
Main loop iteration 1
...
Main loop iteration 10
LED turned off by Timeout.
Main loop iteration 11
...
```

The main loop runs the entire time. The Timeout fires in the background after 2 seconds.

8. Ticker — Periodic Timer

What is it?

Ticker calls a function at a fixed interval, forever, until you detach it. It is interrupt-driven — the callback fires on schedule regardless of what main is doing.

This solves the polling problem from Section 5. Even if main is busy with slow work, the LED still toggles at exactly 500 ms.

When to use it

Use Case	Ticker?
Blink an LED at a precise rate	Yes
Sample a sensor at a fixed frequency	Yes (set flag in ISR, read in main)
Send a periodic heartbeat	Yes
One-time delayed action	Use <code>Timeout</code> instead
Works during deep sleep	Use <code>LowPowerTicker</code> instead

API

```
Ticker t;

t.attach(&my_callback, 500ms); // call every 500ms
t.attach(&my_callback, 1s);    // call every 1s
t.detach();                    // stop
```

Concept Snippet

```
#include "mbed.h"

DigitalOut led(LED1);
Ticker blink_ticker;

void toggle_led() {
    led = !led; // ISR-safe
}

int main() {
    blink_ticker.attach(&toggle_led, 500ms);

    while (true) {
        printf("Main loop running\n");
        ThisThread::sleep_for(1s); // main sleeps, LED still blinks
    }
}
```

Exercise 3 — LED Blinks at 500 ms While Main Prints a Counter

```
#include "mbed.h"

DigitalOut led(LED1);
Ticker blink_ticker;

volatile int tick_count = 0;

void on_tick() {
    led = !led;    // toggle LED – ISR-safe
    tick_count++; // increment counter – ISR-safe (simple write)
}

int main() {
    printf("Ticker demo starting\n");
    printf("LED blinks every 500ms. Main loop runs every 1s.\n\n");

    blink_ticker.attach(&on_tick, 500ms);

    while (true) {
        ThisThread::sleep_for(1s);
        printf("Main loop | Tick count: %d\n", tick_count);
    }
}
```

Expected output:

```
Ticker demo starting
LED blinks every 500ms. Main loop runs every 1s.

Main loop | Tick count: 2
Main loop | Tick count: 4
Main loop | Tick count: 6
Main loop | Tick count: 8
```

Tick count increases by 2 each second because the Ticker fires twice per second (every 500 ms).

9. LowPowerTicker — Periodic Timer Across Sleep

What is it?

LowPowerTicker has the same interface as **Ticker** but runs off LPTIM1/2, clocked from the LSE crystal X2 at 32.768 kHz. It keeps firing even when the CPU is in Stop sleep mode.

A regular **Ticker** requires the main system clock to stay running. With **LowPowerTicker** you can put the CPU to sleep between callbacks and wake it only when needed. On this board that brings current consumption from ~10 mA (active) down to under 10 µA (sleep) — three orders of magnitude.

The trade-off

The LPTIM runs at 32.768 kHz (~30.5 μs per tick):

- Minimum interval is about 1 ms — sub-millisecond intervals are not possible
- Resolution is coarser than a regular **Ticker** (1 μs resolution)
- For sub-millisecond precision, use **Ticker** and accept the CPU stays awake

When to use it

Use Case	LowPowerTicker?
Periodic wakeup on a battery-powered device	Yes
Read a sensor once per second, sleep between reads	Yes
Heartbeat signal in an IoT node	Yes
Sub-millisecond precise timing	Use Ticker instead
One-shot delayed action during sleep	Use LowPowerTimeout instead

API

```
LowPowerTicker t;

t.attach(&my_callback, 2s); // same interface as Ticker
t.detach();
```

Call **ThisThread::sleep_for()** in main to actually enter sleep. Mbed puts the CPU into Stop mode and wakes it when the LPTIM fires.

Concept Snippet

```
#include "mbed.h"

DigitalOut led(LED1);
LowPowerTicker lp_ticker;

volatile bool wakeup_flag = false;

void on_lp_tick() {
    led = !led;
    wakeup_flag = true;
}

int main() {
    lp_ticker.attach(&on_lp_tick, 2s);

    while (true) {
        ThisThread::sleep_for(2s); // CPU enters Stop mode here
    }
}
```

```
        if (wakeup_flag) {
            wakeup_flag = false;
            printf("Woke up\n");
        }
    }
}
```

Exercise 4 — Sleep Between Ticks, Print Timestamp

```
#include "mbed.h"

DigitalOut led(LED1);
LowPowerTicker lp_ticker;
Timer wall_clock;

volatile bool tick_flag = false;
volatile int tick_count = 0;

void on_lp_tick() {
    led = !led;
    tick_count++;
    tick_flag = true;
}

int main() {
    wall_clock.start();
    printf("LowPowerTicker demo starting\n");
    printf("Interval: 2s. CPU sleeps between ticks.\n\n");

    lp_ticker.attach(&on_lp_tick, 2s);

    while (true) {
        ThisThread::sleep_for(2s); // Mbed enters Stop mode here

        if (tick_flag) {
            tick_flag = false;

            auto elapsed_ms = chrono::duration_cast<chrono::milliseconds>(
                wall_clock.elapsed_time()
            ).count();

            printf("Tick %d | Wall time: %d ms | LED: %s\n",
                tick_count,
                elapsed_ms,
                (int)led ? "ON" : "OFF"
            );
        }
    }
}
```

Expected output:

```
LowPowerTicker demo starting
Interval: 2s. CPU sleeps between ticks.

Tick 1 | Wall time: 2003 ms | LED: ON
Tick 2 | Wall time: 4006 ms | LED: OFF
Tick 3 | Wall time: 6009 ms | LED: ON
Tick 4 | Wall time: 8011 ms | LED: OFF
```

Small drift (a few ms) is normal — wakeup latency and LPTIM clock resolution. A regular **Ticker** would stop firing the moment the CPU enters sleep.

Example Use cases

Situation	Use This
Measure how long something takes	Timer
Polling-based periodic task	Timer with <code>elapsed_time()</code> check
Do something once after N ms	Timeout
Do something every N ms	Ticker
Periodic task, CPU can sleep between	LowPowerTicker
One-shot delayed action during sleep	LowPowerTimeout
Measure elapsed time across sleep	LowPowerTimer
Drive a servo or motor	PwmOut
Sub-millisecond precise periodic	Ticker (not LowPower)

Common Mistakes

Mistake	What Goes Wrong	Fix
<code>printf()</code> inside a Ticker callback	Deadlock or corrupted output	Set a volatile flag, print in <code>main()</code>
<code>ThisThread::sleep_for()</code> inside a callback	Hard fault	Never call RTOS functions from an ISR
Forgetting volatile on shared variables	Compiler caches old value, variable appears stuck	Declare ISR-shared variables as volatile
Using Ticker expecting it to work during sleep	Ticker stops when CPU sleeps	Use LowPowerTicker

Mistake	What Goes Wrong	Fix
Long code inside a callback	Misses the next tick	Keep ISR callbacks under ~10 μ s
Forgetting <code>t.start()</code> on a <code>Timer</code>	<code>elapsed_time()</code> always returns zero	Call <code>t.start()</code> before reading

API Quick Reference

Timer	– <code>start()</code> , <code>stop()</code> , <code>reset()</code> , <code>elapsed_time()</code>	no
ISR, polling		
Timeout	– <code>attach(callback, delay)</code> , <code>detach()</code>	ISR,
one-shot		
Ticker	– <code>attach(callback, interval)</code> , <code>detach()</code>	ISR,
repeating		
LowPowerTimeout	– <code>attach(callback, delay)</code> , <code>detach()</code>	ISR,
one-shot, survives sleep		
LowPowerTicker	– <code>attach(callback, interval)</code> , <code>detach()</code>	ISR,
repeating, survives sleep		
LowPowerTimer	– <code>start()</code> , <code>stop()</code> , <code>reset()</code> , <code>elapsed_time()</code>	no
ISR, survives sleep		

11. References

- Mbed OS 6 Time API: <https://os.mbed.com/docs/mbed-os/v6.16/apis/time-apis.html>
- STM32L4 Series Reference Manual (RM0432): In Brightspace
- B-L475E-IOT01A Board User Manual (UM2153): In Brightspace